DSA1605 **Knowledge Engineering and Data Science**
B Nikitha
192373034

Lab Session

1. A class consists of 40 students who have secured different grades in an examination. The grades are classified as F, E, D, C, B, A, and S, where F represents the lowest grade and S represents the highest grade. The number of students who secured each grade is given below.

Grade	Number of Students
F	3
E	4
D	5
C	8
B	9
A	7
S	4
Total	40

Develop a Python program to perform the following tasks:

1. Store the grades and corresponding number of students in suitable data structures.
2. Draw a pie chart to represent the percentage distribution of students across the seven grades.
3. Display the grade labels and percentage values on the pie chart.
4. Draw a bar chart to compare the number of students who secured each grade.
5. Label the X-axis as Grades and the Y-axis as Number of Students.
6. Display the exact number of students above each bar.
7. Add suitable chart titles, legends, and axis labels.
8. Identify the grade secured by the maximum and minimum number of students.
9. Display the percentage of students who secured grades A and S combined.

Aim

To develop a Python program using Matplotlib to represent the distribution of students' grades using a pie chart and a bar chart, identify the maximum and minimum grades, and calculate the combined percentage of students who secured grades A and S.

Procedure

1. Import the matplotlib.pyplot library.
2. Store the grades and corresponding number of students in lists.
3. Calculate the total number of students and the percentage for each grade.
4. Draw a pie chart to represent the percentage distribution of students across all seven grades.
5. Display grade labels and percentage values on the pie chart.
6. Draw a bar chart to compare the number of students in each grade.
7. Label the X-axis as Grades and the Y-axis as Number of Students.
8. Display the exact number of students above each bar and add suitable titles and legends.
9. Identify the grades with the maximum and minimum number of students.
10. Calculate and display the combined percentage of students who secured grades A and S.

Program

```
import matplotlib.pyplot as plt

# 1. Store grades and number of students
grades = ['F', 'E', 'D', 'C', 'B', 'A', 'S']
students = [3, 4, 5, 8, 9, 7, 4]

total = sum(students)

# 2. Calculate percentages
percentages = [(n / total) * 100 for n in students]

# 3. Draw pie chart
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
plt.pie(
    students,
    labels=grades,
    autopct='%1.1f%%',
    startangle=90
)
plt.title('Percentage Distribution of Grades')
plt.legend(grades, title='Grades')

# 4. Draw bar chart
plt.subplot(1, 2, 2)
bars = plt.bar(grades, students, label='Number of Students')

plt.xlabel('Grades')
plt.ylabel('Number of Students')
plt.title('Number of Students in Each Grade')
plt.legend()

# 5. Display exact number above each bar
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        str(int(height)),
        ha='center',
        va='bottom'
    )

plt.tight_layout()
plt.show()

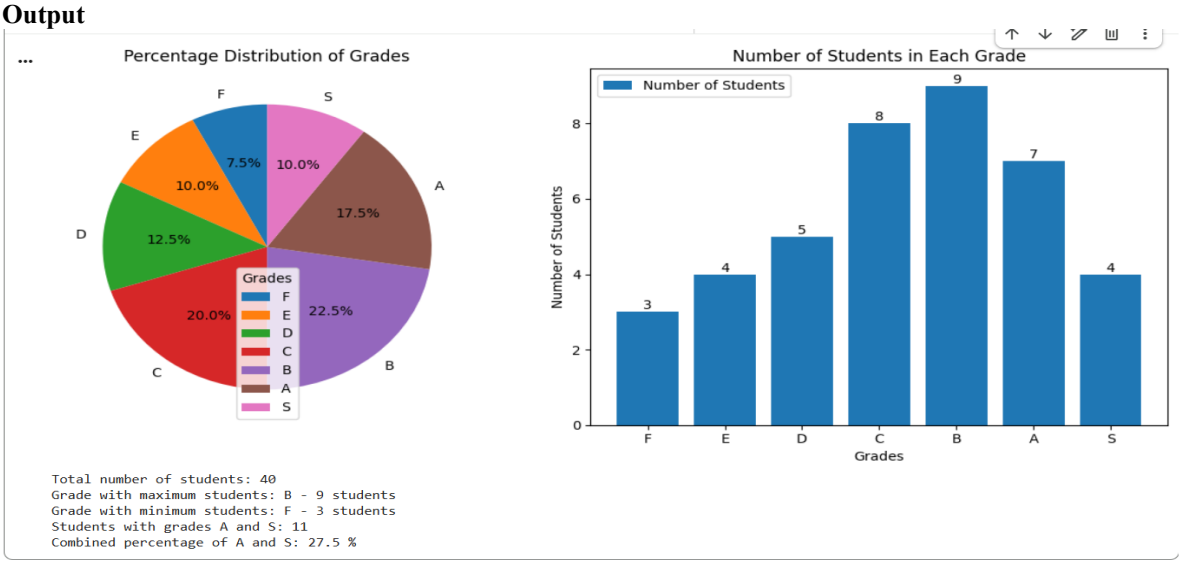
# 6. Find maximum and minimum grades
max_grade = grades[students.index(max(students))]
min_grade = grades[students.index(min(students))]

print("Total number of students:", total)
print("Grade with maximum students:", max_grade,
      "-", max(students), "students")
print("Grade with minimum students:", min_grade,
      "-", min(students), "students")

# 7. Calculate combined percentage of A and S
combined_students = students[grades.index('A')] + students[grades.index('S')]
combined_percentage = (combined_students / total) * 100

print("Students with grades A and S:", combined_students)
print("Combined percentage of A and S:",
      round(combined_percentage, 2), "%")
```

Output



Result

Thus, the Python program was successfully developed and executed to display the pie chart and bar chart representing the grade distribution of 40 students.

- Maximum students: Grade B — 9 students.
- Minimum students: Grade F — 3 students.
- Combined percentage of grades A and S: 27.5%.

2) Semantic Network Using NetworkX

Aim: To represent concepts and labeled relationships as a directed semantic network.

Software Required: Python 3.x and NetworkX (install with: pip install networkx)

Procedure

1. Create a directed graph.
2. Add concepts as nodes implicitly through edges.
3. Add labeled relationships.
4. Display each relationship.

Program

```
import networkx as nx

# Create a directed graph
G = nx.DiGraph()

# Add concepts and labeled relationships
G.add_edge("Dog", "Mammal", relation="is-a")
G.add_edge("Mammal", "Animal", relation="is-a")
G.add_edge("Dog", "Tommy", relation="has-instance")

# Display relationships
print("Semantic Network Relationships:")

for source, target, data in G.edges(data=True):
    print(source, "--", data["relation"], "-->", target)
```

Sample Input

Concepts and relations are predefined in the program.

Output

```
import networkx as nx

# Create a directed graph
G = nx.DiGraph()

# Add concepts and labeled relationships
G.add_edge("Dog", "Mammal", relation="is-a")
G.add_edge("Mammal", "Animal", relation="is-a")
G.add_edge("Dog", "Tommy", relation="has-instance")

# Display relationships
print("Semantic Network Relationships:")

for source, target, data in G.edges(data=True):
    print(source, "--", data["relation"], "-->", target)
```

```
Semantic Network Relationships:
Dog -- is-a --> Mammal
Dog -- has-instance --> Tommy
Mammal -- is-a --> Animal
```

Result: The semantic network represented concepts and their relationships.

3) Frame-Based Knowledge Representation
Aim: To represent an object as a frame containing slots and values.
Software Required: Python 3.x
Procedure
1.Create a dictionary to represent a frame.
2.Use keys as slots and values as slot values.
3. Store the properties of an object in the dictionary.
4.Display the frame name and its slot-value pairs.
5.Execute the program and verify the output.

Program

```
# Frame-Based Knowledge Representation

# Create a frame using a dictionary
frame = {
    "Name": "Tommy",
    "Type": "Dog",
    "Color": "Brown",
    "Age": 3,
    "Sound": "Barks"
}

# Display the frame
print("Frame-Based Knowledge Representation")
print("Frame: Dog")

for slot, value in frame.items():
    print(slot, ":", value)
```

Sample Input
Frame values are predefined in the program.

Output

```
# Frame-Based Knowledge Representation

# Create a frame using a dictionary
frame = {
    "Name": "Tommy",
    "Type": "Dog",
    "Color": "Brown",
    "Age": 3,
    "Sound": "Barks"
}

# Display the frame
print("Frame-Based Knowledge Representation")
print("Frame: Dog")

for slot, value in frame.items():
    print(slot, ":", value)
```

```
Frame-Based Knowledge Representation
Frame: Dog
Name : Tommy
Type : Dog
Color : Brown
Age : 3
Sound : Barks
```

Result: The vehicle was represented using slots and their corresponding values.